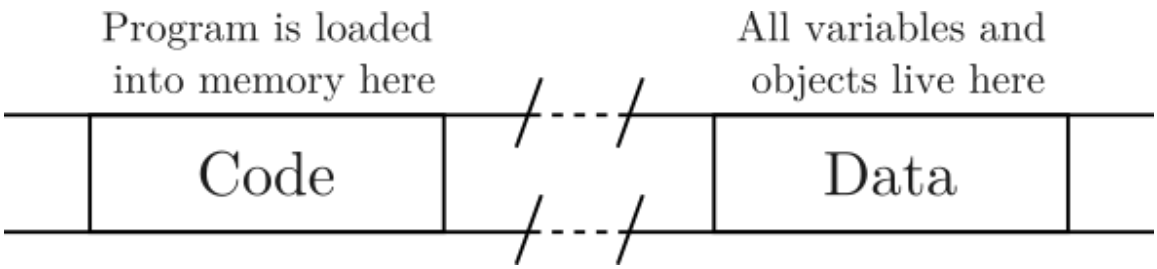
2 - Memory, Arrays and Strings

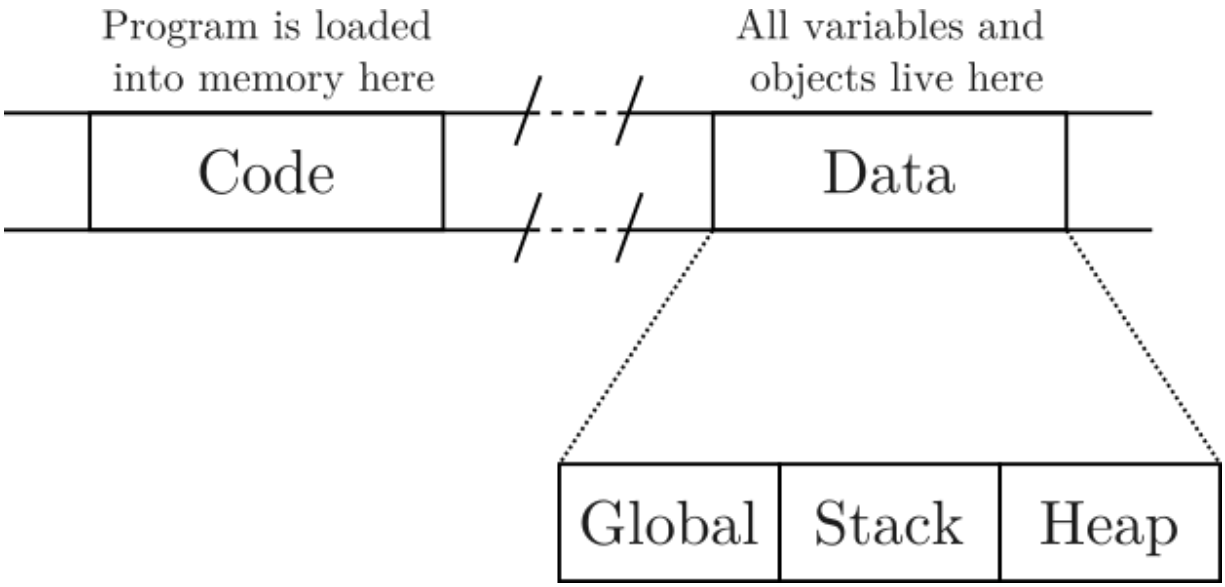
Computers have a variety of technology to retain data. Some of this technology is used to store data when the computer is on while other technology is used to store data when the computer is off. We generally use the term **memory** to refer to short term Random Access Memory (RAM) which is used to store data for running programs and the term **storage** to refer to long term data storage such as Hard Disk Drive (HDD) or Solid State Drive (SSD) which is used to store data long term. Memory supports all of the variables and data in a Java program and it is managed by the programmer and compiler through the Java language and Java virtual machine. To access storage, requires specific system calls to the operating system to make requests to read or write data between memory and a file in storage. For this course, we will entirely focus on memory and while storage will be addressed in future classes.

Memory is like a set of mail boxes servicing a building where every location in the building has an address, *i.e.* a room or apartment number. When a variable is declared in Java, that variable acts as a label to alias a memory address so you the programmer do not need to know or to refer to data by its memory address and instead can refer to it by its label. For a variable of primitive type, the variable stores the value associated with the variable using **direct access** where the memory address aliased by the variable name holds the associated value. For reference types, the variable instead stores an address, *i.e.* the address of the referenced object , which requires a form of **indirect access** where the stored address must first be accessed in one step before a value stored in that object is looked up in a second step. Since references refer to another address we call them **pointers** because they point to the location in memory where the object or container resides.

Program Memory Organization



Computer programs use two blocks of memory to support their operation: the **code block** and the **data block**. Recall, a program is composed of two separate elements: the immutable (read only) code that is executed to run the program, and the mutable (changeable) state data that represents all of the values making up the program at a specific point in execution. The code is stored in the code block and it is not generally possible to change the code during the program's execution. The state data, including all variables and objects associated with the program, is stored separately in the data block.



In Java, the data block is separated into three segments: the **global segment**, the **stack segment**, and the **heap segment**. The scope of a variable depends on which segment the variable exists in and the segment in which a variable lives depends on where (and with what keywords) the variable is declared in the code.

Memory must be explicitly given to a program, and in Java, requests for more memory are built into and managed by the language, compiler, and Java virtual machine. Available, free memory is managed by the operating system and Java makes requests for new memory on behalf of a running Java program according to a prescribed "memory model". When new memory is assigned to a program it is called **memory allocation**.

Given a little experience with Java programming, these memory concepts may seem alien because Java intentionally hides these details from the programmer to simplify the language. Regardless, memory allocation is fundamental to two different Java syntactical elements, *i.e.* declaring a variable or creating a new instance of an object using the `new` keyword.

Memory Allocation

Memory is allocated by the operating system in response to request by a running program. In other words, memory is not allocated until run-time while declaring variables is done at design-time. One of the roles of the compiler is to write allocation rules for every variable declared at compile time. At specific phases during program execution, Java reads the allocation rules for a region of code and handles the allocation request either by using blocks of memory allocated at program start or by passing it through to the operating system for more memory.

In general, memory is either allocated as a **static memory allocation** or as a **dynamic memory allocation**. In this context of memory allocations, the term *static* means "unchanging" and the term *dynamic* means "provided on demand". In other words, a static memory allocation is an allocation made for the program that guarantees that memory allocation will be available for the duration of the program -- note that static does not mean that the value is unchanging but instead means that the allocation is unchanging. In contrast, dynamic memory allocation provides a means for a program to ask for new memory at the moment it needs it. A Java programmer does not have to think too much about these distinctions, but other languages may make the programmer more responsible for memory management and it may be important to discriminate between the two types of allocations. In the following discussion we will plainly identify which variables are subject to these allocation rules.

Java also deallocates memory on behalf of the programmer according to specific deallocation rules so that the programmer does not have to focus on memory management. Declared variables are deallocated when they go out of **scope**, while object are cleaned up by a special process called **garbage collection**. Not all languages provide automatic garbage collection, *e.g.* C and C++, which means that in these languages the programmer is responsible for cleaning up heap memory allocations.

These allocation rules and the underlying organization of memory establish the concept of **variable scope**.

Variable Scope

There are two aspects to a variable’s scope: **lifespan** and **visibility**.

A variable's lifespan is the time between that variable's allocation and its deallocation. A variable is only legally accessible if it has space in memory simply because a variable only exists between the time it is allocated and the time it is deallocated. If there is no space in memory for the variable, the variable does not exist. As such, all variables have a lifespan that dictates when it accessible.

A variable's visibility is the region of code in which that variable is accessible. A variable is only visible as long as the variable exists and the scope of its accessibility is subject to the segment of memory in which it is allocated and is therefore dependent on where it is declared.

There are three variable scopes each of which is associate with a specific data segment: **global variables**, **local variables**, and **instance variables**.

- Global variable - allocated in the global data segment
- Local variable - allocated in the stack data segment
- Instance variable - allocated in the heap data segment

Global Variables

Variables declared inside a class (outside a function) with the `static` keyword are **global variables**, *i.e.* have **global scope**.

```
public class ExGlobal {
    // a, b, c are all global variables
    public static int a;
    public static int b;
    public static int c;

    public static void main (String[] args) {
        a = 5;
        b = 1;
        c = a + b;
    }
}
```

Global variables are stored in the global segment of the data block. All global variables are allocated when the program starts and are deallocated when the program exits and as such their lifespan is the entire run-time of the program. Global variable are accessible anywhere in the program and therefore their visibility is everywhere in the program.

To differentiate global symbols from other symbols, it may be necessary to refer to them by their class name. This is necessary if the global variables are declared in a separate class from the one being operated on. The syntax in Java for accessing a global variable is `<ClassName>.<label>`.

```
public class GlobalVars {
    // d, e, f are all global variables
    public static int d;
    public static int e;
    public static int f;
}

public class ExGlobal2 {

    public static void main (String[] args) {
        GlobalVars.d = 7;
        GlobalVars.e = 8;
        GlobalVars.f = GlobalVars.d + GlobalVars.d;
    }
}
```

Global variables are generally dangerous to use especially in multiprocess applications and should be used for very limited purposes with additional guards used to protect access to them. The reason for this is complex and will be explored in subsequent courses, but the basic problem is that two programs writing data to the same memory location at the same time can cause data to become ambiguous and corrupted in what is called a **data**

race or a **race condition**.

The compiler writes rules for when global variables are allocated and deallocated, so a programmer does not need to generally manage memory for them; however, if a global variable is a reference type, it is necessary to separately consider when the object to which it refers is created.

The global variable being allocated at program start and deallocated at program end coincides with the allocation existing for the entire run time of the program. This meets the definition of a static allocation above and is, in fact, why the Java global variable declaration uses the `static` keyword. In Java, the `static` keyword is used to indicated a static memory allocation and therefore specifies a variable as a global variable.

Local Variables

Variables declared inside a function are **local variables**, *i.e.* have **local scope**.

```
public class ExLocal {
    public static void main (String[] args) {
        // i, j, k are all local variables
        int i = 0;
        int j = 5;
        int k = 100;
    }
}
```

Local variables are stored in the data block's stack segment. Local variables are allocated when the function in which they are declared is called and are deallocated when that specific function call returns. As such, a local variable is only available within the context of that function call, *i.e.* between the opening and closing brackets for the function, and only for that specific function call. Recall that a function can be called many times and can even call itself which means each function call has its own variables even if this results in multiple versions of the same symbol being created.

All local variables use a dynamic allocation model where local variables are dynamically allocated on the stack when the function is called and those same variables are automatically deallocated when the function returns. Local variables are accessible only inside the function after they are declared, but as a rule of thumb, the compiler allocates the local variable when the function is called. For the sake of this class, assume that all local variables for a specific function are allocated when that function is called.

Parameters, *i.e.* input variables declared inside the parentheses of the function declaration, are variables declared in the function, so they are also local variables. Parameters are unique variables, independent in terms of their memory location from the arguments that are passed to the function, so parameters are copies of the original values passed to the function. This policy of copying arguments to parameters is known as **pass-by-value** and is the general policy applied to all function calls in Java and practically it means that changing a parameter in a function will not affect the value of the calling argument. However, this only applies to direct access parameters as any referenced values can be changed inside a function and the change will be reflected outside that function.

The compiler writes rules for when local variables are allocated and deallocated, so a programmer does not need to manage memory for them. In many ways, using the automated allocation and deallocation of local variables can greatly simplify memory management for a program and can ensure that key state data exists throughout the execution of a program.

```
public class Rectangle {
    public int width;
    public int height;
}
```

```
public class GlobalApproach {
    public static Rectangle rectangle;

    public static void main (String[] args) {
        rectangle = new Rectangle();
        ...
    }
}
```

In the `GlobalApproach` program, the global variable `rectangle` has the lifespan of the entire program, so it will be automatically cleaned up when the program exits; however, the global variable actually represents a weakness because we cannot easily limit its visibility to control when and where it can be accessed.

```
public class StackApproach {
    public static void main (String[] args) {
        Rectangle rectangle = new Rectangle();
        ...
    }
}
```

In the `StackApproach` program, the local variable `rectangle` also has the lifespan of the entire program because it is declared and initialized to a legal object at the beginning of `main` and it is on the stack so it will be automatically cleaned up when `main` (the program) exits. The visibility of `rectangle` in this program will be subject to explicitly passing the reference to `rectangle` as an argument.

From a lifespan perspective, the `rectangle` variable has the same behaviors in both programs; however, the visibility in the `StackApproach` program is entirely different. In the `GlobalApproach` program, a programmer cannot control access to the `rectangle` reference, so the object referenced can be accessed, modified, and even destroyed anywhere in the program. In the `StackApproach` program, the `rectangle` reference must be passed to subsequent methods or objects to gain access to the referenced object; however, this will be done explicitly and within any rules created by the developers.

The `StackApproach` example represents a way to control and limit access to an object that is needed program-wide while also gaining similar memory management as a global variable.

Instance Variables

Variables declared inside a class (outside a function) without the `static` keyword are **instance variables**, *i.e.* have **instance scope**, and are associated with (belong to) a specific instance of an object.

```
public MyObject {
    // x, y, z are all instance variables
    public int x;
    public int y;
    public int z;
}

public class ExInstance {

    public static void main (String[] args) {
        // obj is a new instance of MyObject
        // class. When obj is created, new
        // instances of all instance variables
        // declared in the class are created.
        MyObject obj = new MyObject();
        obj.x = 0;
        obj.y = 2;
        obj.z = 50;
    }
}
```

Instance variables are stored in the data block's heap segment. All instance variables declared in an object's class definition are allocated for an object when that object is newly created from a class and all instance variables for that object are available as long as there is a valid reference to that object. A reference to an object can be a global variable, a local variable, or even an instance variable belonging to another object.

Instance variables are accessible through a valid reference to the object using the syntax `<reference-label>.<instance-label>`. In the `ExInstance` example above, `obj` is declared as a reference type to objects of type `MyObject`. While `obj` lives on the stack, it is a local variable declared in `main`, the object of type `MyObject` to which it refers lives on the heap. To access the instance variables inside the object referred to by `obj`, the programmer must use the `obj` label as a prefix. In other words, `obj.x` accesses the instance variable `x` stored inside the object referred to by the instance variable `obj`. Instance variables can only be accessed outside the object using a valid reference to an object.

Instance variables do not exist until an object instance is explicitly created. The `new` keyword asks Java to allocate a new instance of an object with the type that follows the `new` keyword. All non-primitives must be allocated with the `new` keyword. The `new` keyword causes the run-time environment to create the object on the heap. All objects live on the heap.

All instance variables use a dynamic allocation model where instance variables are dynamically allocated on the heap when the object to which they belong -- the instance -- is constructed. Instance variables are accessible inside the object and are available as long as the object exists. An object exists until there are no more references to the object. The Java system automatically deallocates an object if it no longer has any references to it using garbage collection.

```
public MyObject {
    // x, y, z are all instance variables
    public int x;
    public int y;
    public int z;
}

public class ExInstance {

    public static void main (String[] args) {
        MyObject obj2;
        ...
        obj2 = new MyObject();
        obj2.x = 0;
        obj2.y = 2;
        obj2.z = 50;
    }
}
```

Remember that the reference to the object and the object are two separate things stored independent of one another in memory. The above example hopes to illustrate that the declaration of `obj2` does not require or even guarantee that there is an object associated with it. It is legal for a reference to point to nothing or `null`. Creation of the object is an explicit step that requires usage of the `new` keyword.

Garbage Collector

If an object on the heap has no references, then it cannot be accessed anywhere in the program and the object needs to be deleted. Programming languages may use several strategies as to how to resolve deallocation of heap memory.

Some languages put the burden of deallocating and releasing heap memory back to the system on the programmer. For example, C requires programmers to free the last reference to an object before the reference is set to null (or the program exits). If the programmer does not free all heap memory, then the memory can stay allocated even after a program exits. This phenomena is called a memory leak and explains why some programs can hoard resources from the operating system and other programs even if the program exits which can only be resolved by a reboot.

Java as well as many other high level languages provide a process that prowls around the heap looking for objects that are inaccessible. Rather than forcing the programmer to manage heap memory, this process takes on the role of deallocating heap memory if an object has no more references. This process is called the garbage collector which performs garbage collection on the heap. Note that garbage collection is only performed on the heap as

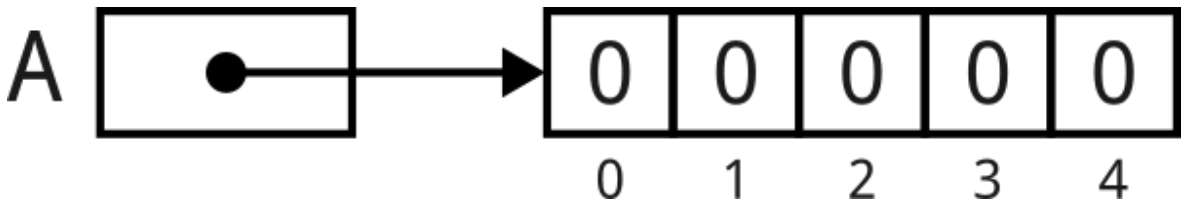
the stack and global segments allocate and deallocate using different rules.

The garbage collector comes at a cost. It runs about every millisecond and steals some processing time away from the processes that you are actively running. If there is a significant amount of deallocation work, the garbage collector will interrupt progress of any process while it cleans up the heap. This can be a problem with real-time applications that must meet deadlines such as control software for planes or robots. In the case of these applications, it may be better to choose a language that forces the programmer to control deallocations because the programmer also controls when deallocations occur and there is not an uncontrollable process that may interrupt scheduled events.

Arrays

An array is a set of memory intended to hold a set of elements all with the same type. Java uses special syntax for arrays however Java arrays are still classes. Therefore, Java arrays have properties and methods accessible by the dot operator and Java array declarations are also non-primitive references.

```
1  public class ArrayEx {
2
3      public void main(String[] args) {
4          int[] A;        // A is declared as a reference type that will refer
5                          // to an array of ints
6          ...
7          A = new int[5]; // the new keyword requests creation of an array of ints
8                          // with 5 elements.  Up until this point, A referred to
9                          // null (nothing).  After line 7, A will refer to the
10                         // array allocated by line 7
11      }
12 }
```



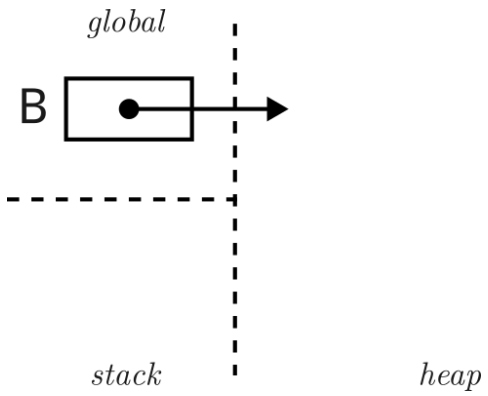
As a result, the array is an entirely separate entity from the variable declared in code. A programmer must consider the scope of both the array reference pointer and the array object independently. These two entities may live in entirely different memory locations, where the location of the array reference is subject to where its variable is declared and the array itself always lives on the heap because it is an object.

An object may have multiple references to it, so while the pointer initially assigned the reference to a new object may be cleaned up, one or more additional pointers to the object may have been established between object creation and initial pointer deletion which means the object itself will not be cleaned up until all references to the object go out of scope.

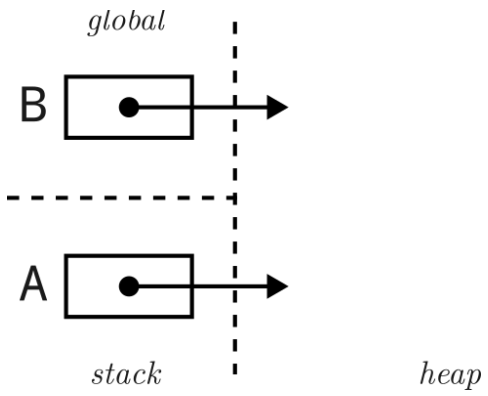
```
1  public class MultiPointerEx {
```

```
    public static int[] B;

    public void main(String[] args) {
        foo();
        ...
    }

    public void foo() {
        int[] A;        // A is local to foo.
        ...
        A = new int[5]; //  A holds initial reference to array
        ...
        B = A;          // B is a global var.  B copies the reference to A
        ...
        // A and B have different scopes.  A is cleaned up upon return from foo.
        // B is global and will persist even after foo.  As a result, the array
        // will persist even after A goes out of scope here because there is at
        // least one reference to it beyond A in this case via B.
    }
}
```

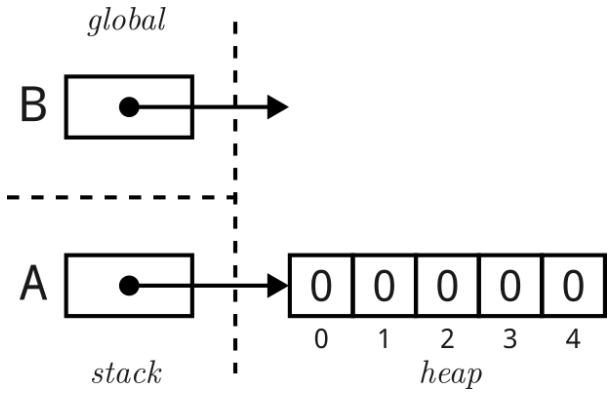
The `MultiPointerEx` example demonstrates copying a pointer. Regardless of the type that is referenced, all reference types contain only a memory addresses. In other words, the line `B = A;` copies the address stored in `A` into `B`. It does NOT copy the data stored in the object into a new object. In the `MultiPointerEx` there is only one object, but inside `foo()`, there are temporarily two references to that object. At the end of `foo()` the global variable `B` continues to refer to and anchor the object in heap memory even after variable `A` is deallocated because the `foo()` returns and all of its local variable go out of scope.



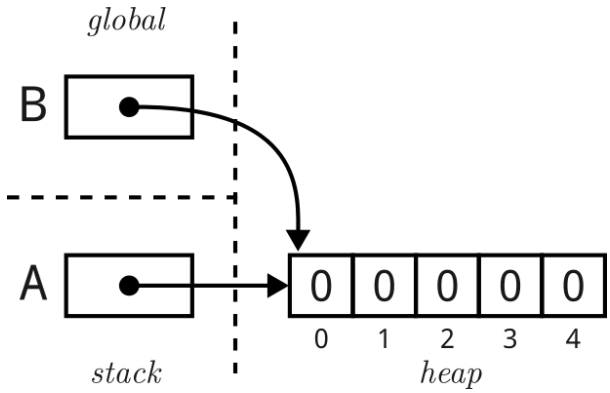
In the `MultiPointerEx` example above, `B` is declared as global variable, so it is allocated in the global area when the program starts. However, it is not assigned to an object so by the rules of Java as a reference type it is initially assigned to point to nothing.



When the method `foo()` is called, the local variable `A` is allocated on the stack; however, like variable `B`, variable `A` has not been assigned an initial value, so it too is assigned to initially point to nothing.



At line 12, variable `A` is assigned to point to (a reference to) the newly allocated array of integers with 5 elements. As the array is a container type/object, it is allocated on the heap because all objects live on the heap.



At line 14, `B` is assigned the same reference value stored in `A`. Note that this does not copy the array, it only copies the reference to the array. As a result, both `A` and `B` point to the same array.

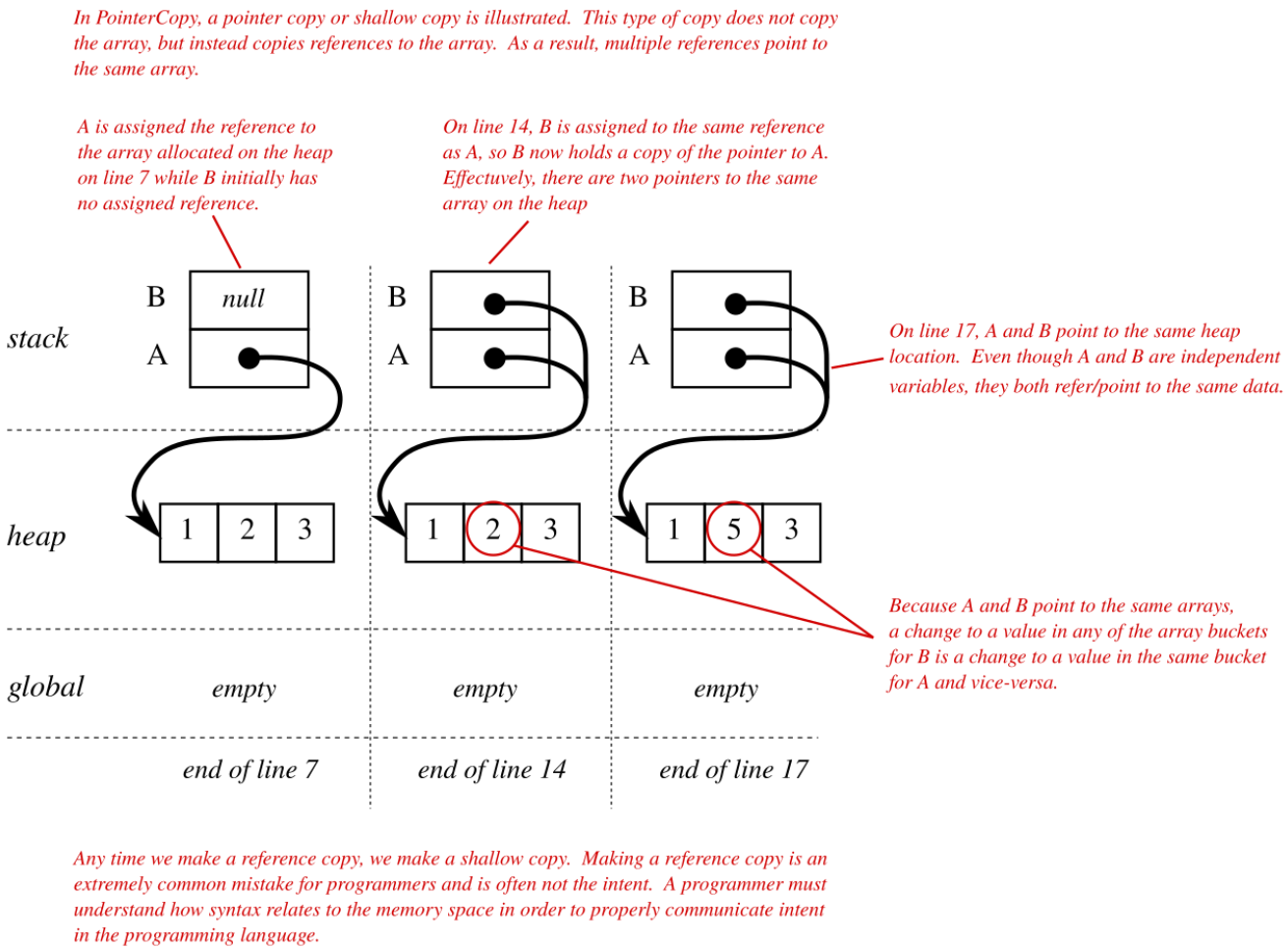
Copying a pointer does not create new objects. Instead it copies the address of the object. This means that a programmer must explicitly develop a means to copy objects if copying objects is the programmer's intent. In contrast, copying a primitive is simple and straightforward using the assignment operator. Copying a pointer may be called a shallow copy and copying a primitive value may be called a deep copy.

Shallow Copy and Deep Copy

We use the simple terms **shallow copy** and **deep copy** to communicate what is copied. The term shallow copy tries to communicate a superficial or surface level copy of information and the term deep copy tries to communicate a comprehensive or deep level copy of information.

The following `PointerCopy` example demonstrates a shallow copy where two references refer to the same object in memory. Since both references refer to the same data, a change to either pointer is visible to both.

```
import java.util.*;

public class PointerCopy {

    public static void main (String[] argv)
    {
        int[] A = {1, 2, 3};
        System.out.println ("A: " + Arrays.toString(A));

        // Declare a new array variable.
        int[] B;

        // Pointer copy!
        B = A;

        // A modification of B affects A
        B[1] = 5;
        System.out.println ("A: " + Arrays.toString(A));
    }
}
```

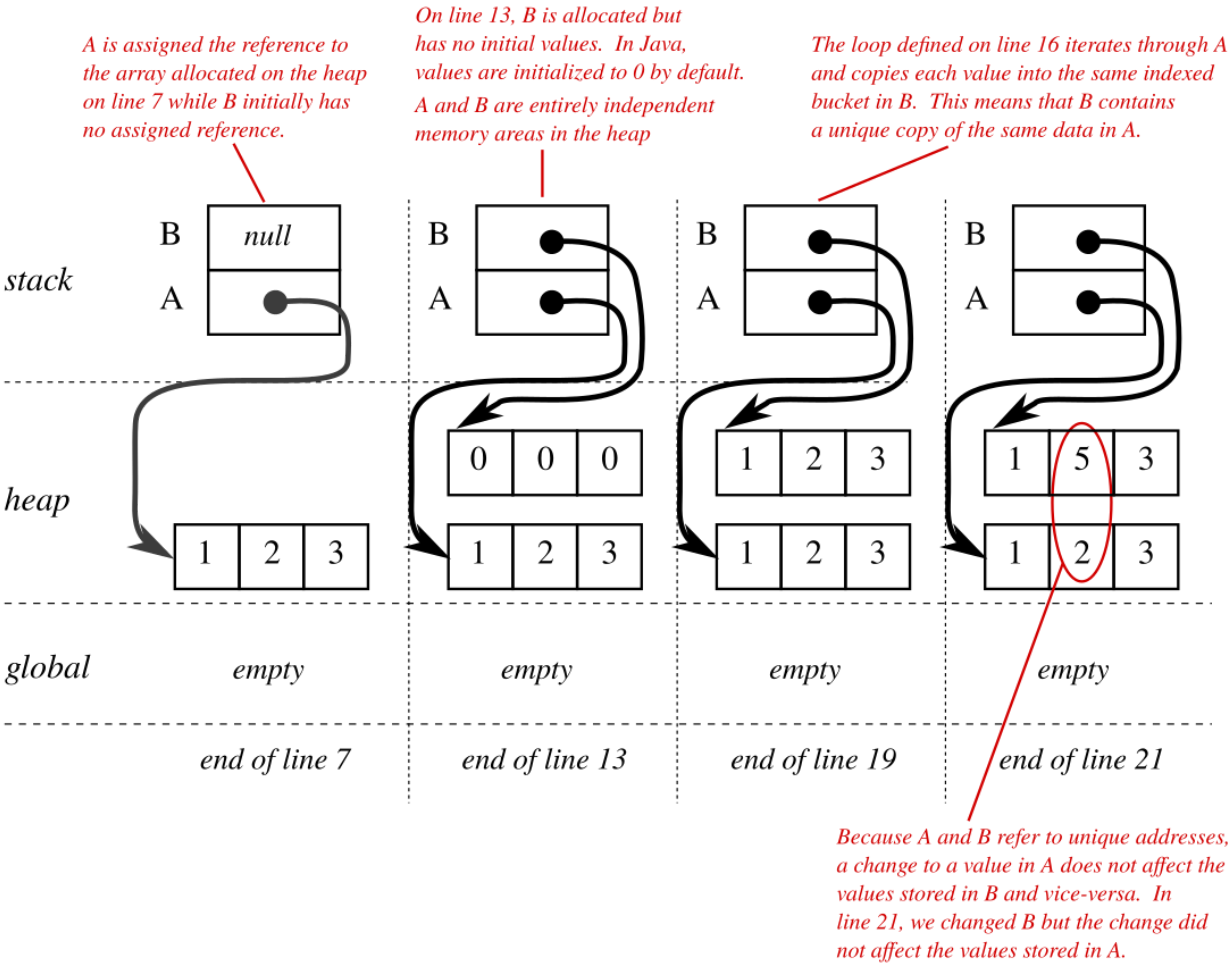


The following `ValueCopy` example demonstrates a deep copy where both references point to independent arrays of the same size. The data is copied from one array to the other and while the second array contains duplicate information, it is independent of the other array after the copy operation is completed. Since the references are refer to different memory locations, a change to either one does not affect the other.

```
1  import java.util.*;
2
3  public class ValueCopy {
4      public static void main (String[] argv)
5      {
6          int[] A = {1, 2, 3};
7          System.out.println ("A: " + Arrays.toString(A));
8
9          int[] B;
10
11         // First make the space - getting the length from A.
12         B = new int [A.length];
13
14         // Copy each entry.
15         for (int i=0; i < A.length; i++) {
16             B[i] = A[i];
17         }
```

```
18
19          // Modification of B does not affect A.
20          B[1] = 5;
21          System.out.println ("A: " + Arrays.toString(A));
22      }
23 }
```

In ValueCopy, a value copy or deep copy is illustrated. This type of copy copies every value in the array to another array. Each array is a unique and independent entity and therefore changes to one do not affect the other.



Any time we make a value copy, we make a deep copy. Often, when we use the term "copy" we implicitly mean deep copy; however, we have to convey the intent to our programming language as they typically support both shallow and deep copies. Both shallow and deep copies are very useful tools, so it is important that both are supported; however, because we have both tools, programmers must convey in the programming language which is intended.

Strings

The term **string** is used in computing to describe a sequence of characters. Java provides a `String` class to encapsulate all properties and methods associated with strings into a single class. While the Java `String` class appears quite complex at least with respect to [having about a hundred methods associated with it](), the data in the class is simply just an array of `char` values. In other words, the `String` class is just a wrapper for a `char[]` property and any associated string operations.

ASCII

The American Standard Code for Information Interchange (ASCII) is a convention for character encoding first established in the 1960's. ASCII defines the association between character data and the numeric values that represent them. Recall that all data in memory is just a number, so the role of ASCII is to provide a translation layer between the numeric representations of `char` data in memory to the symbolic letters used in English.

For example, the decimal value of 65 is interpreted and translated as the English character symbol 'A' if it is typed as character data. Similarly, the decimal value of 66 translates to the character symbol 'B' and so forth.

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	00	NUL	32	20	SPACE	64	40	@	96	60	`
1	01	SOH	33	21	!	65	41	A	97	61	a
2	02	STX	34	22	"	66	42	B	98	62	b
3	03	ETX	35	23	#	67	43	C	99	63	c
4	04	EOT	36	24	$	68	44	D	100	64	d
5	05	ENQ	37	25	%	69	45	E	101	65	e
6	06	ACK	38	26	&	70	46	F	102	66	f
7	07	BELL	39	27	'	71	47	G	103	67	g
8	08	BS	40	28	(	72	48	H	104	68	h
9	09	TAB	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	|
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

If the string "HELLO" is stored as a Java `String` in memory, what is really stored is an array of `char` data with the values 72, 69, 76, 76, 79 where 72 is 'H', 69 is 'E', 76 is 'L', 76 is 'L', and 79 is 'O' according to the ASCII standard.

Char	H	E	L	L	O
ASCII	72	69	76	76	79
Index	0	1	2	3	4

Each block of uppercase and lowercase letters are in alphabetical order. Additionally the difference between 'A' and 'a' is exactly 32 (2^5), so to convert from uppercase to lowercase is just addition and to convert from lowercase to uppercase is just subtraction.

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	00	NUL	32	20	SPACE	64	40	@	96	60	`
1	01	SOH	33	21	!	65	41	A	97	61	a
2	02	STX	34	22	"	66	42	B	98	62	b
3	03	ETX	35	23	#	67	43	C	99	63	c
4	04	EOT	36	24	$	68	44	D	100	64	d
5	05	ENQ	37	25	%	69	45	E	101	65	e
6	06	ACK	38	26	&	70	46	F	102	66	f
7	07	BELL	39	27	'	71	47	G	103	67	g
8	08	BS	40	28	(	72	48	H	104	68	h
9	09	TAB	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	|
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

Uppercase Letters 'A' (65)- 'Z' (90)

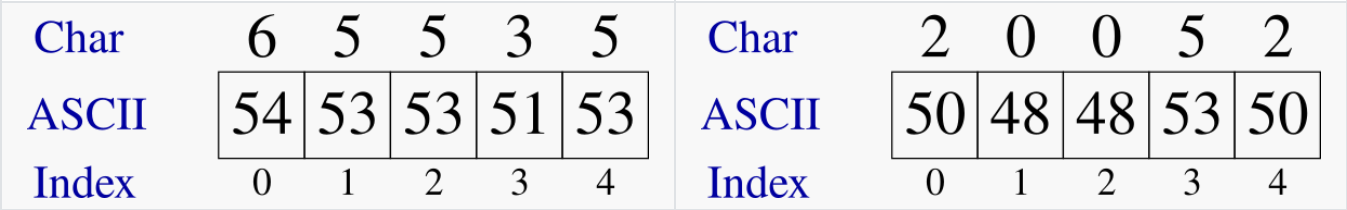
Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	00	NUL	32	20	SPACE	64	40	@	96	60	`
1	01	SOH	33	21	!	65	41	A	97	61	a
2	02	STX	34	22	"	66	42	B	98	62	b
3	03	ETX	35	23	#	67	43	C	99	63	c
4	04	EOT	36	24	$	68	44	D	100	64	d
5	05	ENQ	37	25	%	69	45	E	101	65	e
6	06	ACK	38	26	&	70	46	F	102	66	f
7	07	BELL	39	27	'	71	47	G	103	67	g
8	08	BS	40	28	(	72	48	H	104	68	h
9	09	TAB	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	|
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

Lowercase Letters 'a' (97)- 'z' (122)

Similarly all numeric characters are also grouped together and in order beginning with 48 is '0' through 57 is '9'.

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	00	NUL	32	20	SPACE	64	40	@	96	60	`
1	01	SOH	33	21	!	65	41	A	97	61	a
2	02	STX	34	22	"	66	42	B	98	62	b
3	03	ETX	35	23	#	67	43	C	99	63	c
4	04	EOT	36	24	$	68	44	D	100	64	d
5	05	ENQ	37	25	%	69	45	E	101	65	e
6	06	ACK	38	26	&	70	46	F	102	66	f
7	07	BELL	39	27	'	71	47	G	103	67	g
8	08	BS	40	28	(	72	48	H	104	68	h
9	09	TAB	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	|
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

It is important to understand that if numeric information has a `char[]` or `String` type, it is really represented as an array of characters in ASCII format and these values will require a translation step to convert them into numeric types compatible for math.



The first column (0-31) in the ASCII table contains 'control codes' which are generally unprintable or whitespace characters that are easy to overlook but can play crucial roles in documents.

For example, the character 9 is the 'TAB' character which represents the whitespace Tab character associated with the Tab key on the keyboard. Note that 'TAB' is distinct from character 32 which represents the 'SPACE' character which represents the whitespace Space character associated with the Space key on the keyboard. These characters are distinct and are treated different due to the historical roles they played on typewriter keyboards.

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	00	NUL	32	20	SPACE	64	40	@	96	60	`
1	01	SOH	33	21	!	65	41	A	97	61	a
2	02	STX	34	22	"	66	42	B	98	62	b
3	03	ETX	35	23	#	67	43	C	99	63	c
4	04	EOT	36	24	$	68	44	D	100	64	d
5	05	ENQ	37	25	%	69	45	E	101	65	e
6	06	ACK	38	26	&	70	46	F	102	66	f
7	07	BELL	39	27	'	71	47	G	103	67	g
8	08	BS	40	28	(	72	48	H	104	68	h
9	09	TAB	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	|
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

Understanding typewriters and teletype systems can offer some interesting perspective on the ASCII model and the artifacts programmers must deal with even today. For example, the end of line on a typewriter was a mechanical operation involving multiple individual actions. When the end of line was reached:

1. The operator would return the carriage to its left most position by striking the carriage handle. This is known as carriage return.
2. Upon carriage return, the drum, around which the paper was half looped, would feed to the next line. This is known as line feed.

By GodeNehler - Own work, CC BY-SA 4.0

Memorialized in ASCII are the carriage return (CR) as decimal value 13 and line feed (LF) as decimal value 10. Since programmers are not constrained by the mechanics of a typewriter, different platform developers did not feel the need to fully model the typewriter's operation to represent end of line. This resulted an interesting and fundamental difference in file data on Windows, Mac, and Linux when it comes to interpretation of the end of line (EOL) character, *i.e.* `\n`. On Windows, EOL is represented as the sequence of carriage return followed by linefeed, so on Windows CRLF is embedded in text when an EOL is translated to `char` data. On Mac systems before the OSX version was adopted, the EOL character embedded CR only. On Linux systems and Mac OSX systems, the EOL character is LF only.

Pangram

Word games are common in languages and one game asks a simple question of whether or not a phrase contains all symbols from a language. This is known as a *pangram* derived from *pan* meaning all and *gram* meaning symbols. The most common example of an English pangram is “The quick brown fox jumps over the lazy dog” which raises the question of how can we determine whether or not a phrase is a pangram.

The brute force pangram checking algorithm is the obvious one that most people adopt when posed the problem of a pangram. It generally follows the approach:

```
# alphabet : the alphabet to for all letters of
# phrase : the phrase to check to see if it is a pangram
# returns : true if phrase contains all letters in alphabet, otherwise, false
ispangram-bf(alphabet, phrase):

    1. Beginning with the alphabet letter 'A' up to 'Z'
       1.1 For each letter in the phrase
           1.1.1 If the alpha letter equals the phrase letter
               1.1.1.1 Found the alpha letter
               1.1.1.2 Can go to next alpha letter
       1.2 If alpha not found
           1.2.1 The phrase is not a pangram
           1.2.2 Stop return false
    2. 'A' - 'Z' must have been found to get here, so must be a pangram return true
```

While we are dealing with a rather narrow problem that will not generally be inefficient for a language like English with only 26 letters, we might want to use a similar strategy in other symbolic languages. Consider if we want to play this same game with traditional Chinese which has well over 10,000 unique symbols. Maybe instead the question becomes, "are all symbols in the language used in this document". While the English pangram can use brute force because of the small amount of symbols in the language, the same algorithm becomes much slower in another application. Can we write a more efficient algorithm for this process?

First consider, the claim that the brute force approach to pangram is slow. Why is it slow and how can it be improved? The inefficiency exists because the process starts over at the beginning of the phrase for each symbol. In essence, the outer loop structures the question as look at every letter in phrase for 'A', then do the same for 'B', etc. This means that in the worst case, it takes two entire passes through the phrase in the inner loop to determine that both 'A' and 'B' exist in the phrase. For the entire alphabet, it would take 26 passes. For English, again this is not so slow but for a language like traditional Chinese this process is considerably slower. Is it possible to do this in only one pass through the phrase?

It is possible if we reconsider the fundamental question we ask. Instead of repeatedly asking each letter in the phrase whether that letter is an 'A' and then doing the same with each subsequent alphabet character, why not ask each letter in the phrase "what letter are you" and then track the answers somehow. Perhaps we can use an array to represent the count of each letter we find in the phrase. Consider the following alternative algorithm

```
# alphabet : the alphabet to for all letters of
# phrase : the phrase to check to see if it is a pangram
# returns : true if phrase contains all letters in alphabet, otherwise, false
ispangram-opt(alphabet, phrase):

    1. Create an array of len(alphabet) integers indexed 0 to len(alphabet) - 1
       with initial values of 0

    2. For each letter in the phrase
       2.1 positon = letter - alphabet[0]
       2.2 array[position] = array[position] + 1

    3. For each value in array
       3.1 if value is 0
           3.2 phrase is not a pangram, return false

    4. phrase is a pangram, return true
```

`ispangram-opt` loops through the phrase once. It's weakness is it does loop through the alphabet once as well which is actually unnecessary with a slight optimization. This approach is faster than the brute force technique initially proposed because it eliminates a significant number of operations contributed by using a nested loop structure.

There is one open question that arises from the `ispangram-opt` pseudocode above , specifically about how to compute `positon = letter - alphabet[0]` in a strongly typed language like Java. Consider that `letter` and `alphabet[0]` are characters yet `position` is an integer, so the question is how to overcome Java's protection against performing math on a non-math type like a `char` and the answer lies in type casting.

Casting Types

Recall that Java is a strongly typed language which means that all variables must be assigned a type which is done through variable declaration. In fact, all values in Java have a type, this includes all literals. A literal is a "hardcoded", constant value.

For example, the symbols `0`, `1.0`, `false`, `"Hello"`, and `null` are all literals and they all carry with them implicit types. `0` or any counting number is generally interpreted to be an `int` type, `1.0` or any number containing a decimal is generally interpreted to be a `float` type, `false` and `true` are generally interpreted to be of `boolean` type, anything bounded by a double quotes, *e.g.* `"Hello"`, is generally interpreted as a `String` literal, and the `null` value is interpreted to be a reference type.

There are a variety of reasons to cast types and we can cast primitive values and reference types. Later in the course we will see reference types change depending on their declarations and the relationships with other types. Primitives might be cast because we have a `char` type but we need to cast it to an `int` to perform a math operation on the data or we might cast an `int` to a `char` after performing some transformation on the data and

we need to use the value in string operations. There are other cases to cast as well, but they are beyond the context of this discussion.

```
String phrase =“the quick brown fox jumps over the lazy dog”;
char[] carr = phrase.toCharArray();
char ch = carr[0];  // ch is a copy of the first character in phrase
int ascii = (int) ch; // ascii is the integer code of ch
```

Recall that all values in memory are just numbers and the type acts as a filter that determines how the values are presented to the user. In the above example, the `ascii` variable contains a copy of the value stored in `ch`; however, the `int` type associated with the `ascii` variable .

Suppose we want to find the position of a character in the alphabet. The position of a letter in the alphabet is basically the distance of that letter from the letter 'a'. In ASCII, the alphabet is logically ordered and every character has a numeric value, so it is just a matter of expressing to Java that your intent really is to perform math on letters even if it is a non-standard thing to do.

```
char ch = carr[0];  // ch is a copy of the first character in phrase
int ascii = (int) ch; // ascii is the integer code of ch
int pos = ascii - (int)'a';
```

In the above example, type casting is used to clarify two things: that the conversion of `ch` from character type to `int` type is intentional and to cast the value of `a` into a type compatible with the subtraction operation and that this statement is intentional.

Review

1. What is stored in the program block and what is stored in the data block?
2. The data block is divided into three segments. What are these three segments?
3. Define variable scope.
4. What are the three scoping levels associated with Java?
5. What is the lifespan of a variable and how does variable lifespan relate to variable scope?
6. What is the visibility of a variable and how does variable visibility relate to variable scope?
7. What is the lifespan and visibility of a global variable?
8. What is the lifespan and visibility of a local variable?
9. What is the lifespan and visibility of an instance variable?
10. How does the scope of a variable relate to the memory allocation of that variable?
11. How does the declaration of a variable relate to its scope?
12. Given the following code snippet, explain what is copied on line 2?

    ```
    int[] A = new int[10];
    int[] B = A;
    ```

13. What is an array?
14. What is a String?
15. What is ASCII?
16. What does it mean to cast from one type to another?
17. Why is it possible to cast from one type to another?
18. How do you cast from a character type to an integer type in Java? In other words, what is the syntax and give an example.
19. How do you cast from an integer type to a character type in Java? What is the syntax and give an example.
20. Explain what is meant by the term “deep copy”.
21. Explain what is meant by the term “shallow copy”.
22. Contrast the idea of a deep copy with a shallow copy.
23. Give an example in Java of a shallow copy.
24. Give an example in Java of a deep copy.
25. When a primitive value is copied, is that a deep copy or a shallow copy?
26. When a reference value is copied, is that a deep copy or a shallow copy?
27. If a deep copy of an array is made and a change is made to the copied array, how is the original array affected? Why?
28. If a shallow copy of an array is made and a change is made to the copied array, how is the original array affected? Why?
29. Illustrate the memory allocation Java would make for the following statement. Assume the statement appears inside a function.

    ```
    int[][][] p = new int[4][3][2];
    ```

30. Illustrate the memory allocation Java would make for the following statement. Assume the statement appears inside a function.

    ```
    int[][] q = new int[4][3];
    ```

Vocabulary

memory

storage

primary memory

secondary memory